

Part 2: Dictionary attacks

Ethical and authorized use: run these scripts only against the fictional accounts and local files supplied with this lab. Do not use leaked wordlists, real credentials, or external services. The SSH extension may target only the fixed lab01-ssh-target Compose service. It must never be modified to accept an IP address, public hostname, different username, or external wordlist.

The CSV schemas are `account,sha256` and `account,salt_hex,sha256`. Complete both starters, then run `python3 compare_cost.py`. Report recovered synthetic accounts, computation counts, elapsed time, and guesses per second.

Purpose and learning objectives

This part changes perspective from verifier construction to offline guessing. Assume an authorized tester has already received a small hash file. Because no server is contacted, every candidate can be checked locally and server-side lockout or rate limiting does not apply.

After completing this part, you should be able to:

- implement a transparent dictionary attack using `hashlib` and `csv`;
- reuse one candidate hash across all unsalted accounts;
- recompute a candidate separately for each unique salt;
- count hash operations and measure elapsed time correctly; and
- explain the protection salts provide and their limitations.
- distinguish offline hash guessing from observable online login attempts.

Data model and attack logic

`unsalted_hashes.csv` uses this schema:

```
account,sha256
```

For each word, compute `SHA256(UTF8(word))` once and use the digest as a lookup against all target rows.

`salted_hashes.csv` uses this schema:

```
account,salt_hex,sha256
```

For each account and word, decode the salt and compute `SHA256(salt || UTF8(word))`. A digest computed for one account cannot be reused for an account with a different salt.

Unsalted work: approximately `number_of_words` hash operations
Salted work: up to `number_of_accounts * number_of_words` operations

The exact count may be lower if your implementation stops checking an account after finding its candidate. State your stopping rule when comparing results.

Tasks

1. Inspect the wordlist and both CSV schemas; do not edit their target values.
2. Implement `crack()` in `crack_unsalted.py` using one hash per candidate.
3. Return (`recovered`, `computation_count`, `elapsed_seconds`) as documented.
4. Implement `crack()` in `crack_saltd.py`, decoding each hexadecimal salt.
5. Run both programs separately and confirm they recover fictional accounts.
6. Run `compare_cost.py` and compare computation counts, not only wall time.
7. Explain how the cost would scale to 1,000 accounts and a larger dictionary.
8. Complete the controlled SSH extension and compare its online behavior with the two offline attacks.

Inspect the local inputs

```
cd /workspace/labs/lab01/part2_dictionary_attack
head ../data/lab01-small.txt
sed -n '1,5p' ../data/unsalted_hashes.csv
sed -n '1,5p' ../data/salted_hashes.csv
```

The wordlist contains only fictional classroom candidates. In the salted CSV, decode `salt_hex` with `bytes.fromhex(...)` before concatenating it with the UTF-8 candidate bytes.

Python files and usage examples

Both attack files are student starters. Before implementation, their commands intentionally stop at `NotImplementedError`.

`'crack_unsalted.py'`

This program accepts the unsalted target CSV followed by the wordlist:

```
python3 crack_unsalted.py ../data/unsalted_hashes.csv ../data/lab01-small.txt
```

After completing `crack()`, its output should have this shape:

```
({'fictional_account': 'recovered_candidate'}, HASH_COUNT, ELAPSED_SECONDS)
```

Hash every candidate once and compare it with all target digests.

`'crack_salted.py'`

This program accepts the salted target CSV followed by the same wordlist:

```
python3 crack_salted.py ../data/salted_hashes.csv ../data/lab01-small.txt
```

Its output uses the same tuple format:

```
({'fictional_account': 'recovered_candidate'}, HASH_COUNT, ELAPSED_SECONDS)
```

Compute a separate digest for every candidate/account salt pair. Do not hard-code account names, candidates, counts, or digests in either program.

`'compare_cost.py'`

After both attack functions work, run them together and print a compact table:

```
python3 compare_cost.py ../data
```

Expected output format (numbers depend on your implementation and machine):

experiment	found	hashes	seconds	hashes/s
unsalted	N	N	0.000000	N
salted	N	N	0.000000	N

Controlled online SSH dictionary exercise

This extension uses a deliberately vulnerable SSH server in a separate Docker container. It is an online attack: every candidate opens an SSH connection, and the server can see, log, delay, or reject the attempt. The target is not published to the host and is connected only to an internal Compose network.

Security boundaries

- The only permitted target is labstudent@lab01-ssh-target:22.
- The only permitted input is ../data/ssh-lab-wordlist.txt (at most 20 lines).
- Do not add target, port, username, concurrency, or external-wordlist options.
- Do not publish a port from the SSH target to the host.
- Stop and remove the target container after the exercise.

Step 1 - Build and start the isolated target

Run these commands from the repository root in a host terminal, not from inside the course container:

```
docker compose -f docker/compose.yml --profile lab01-ssh build
docker compose -f docker/compose.yml --profile lab01-ssh up -d lab01-ssh-target
docker compose -f docker/compose.yml --profile lab01-ssh ps
```

The target should become healthy. There is deliberately no ports: mapping for this service, so host and external clients cannot connect directly.

Step 2 - Enter the course container on the lab network

```
docker compose -f docker/compose.yml --profile lab01-ssh run --rm course bash
cd /workspace/labs/lab01/part2_dictionary_attack
```

Step 3 - Review and complete 'ssh_dictionary_attack.py'

The constants fix the destination to the course service. Implement only try_candidate():

1. create paramiko.SSHClient();
2. use AutoAddPolicy only because this target is an isolated, disposable classroom container;
3. call connect() with the fixed host, port, and username;
4. disable agent and local-key lookup so only the candidate is tested;
5. return False for paramiko.AuthenticationException;
6. return True after successful authentication; and
7. close the client in a finally block.

Do not print candidate strings. The provided attack loop is sequential, waits between failures, and reports only status, attempt count, and elapsed time.

Run the starter checks:

```
pytest -q test_ssh_dictionary_attack.py
```

Expected result: 2 passed. These checks confirm the target is fixed and the course wordlist is bounded; they do not reveal the implementation or password.

Step 4 - Run the bounded online attack

```
python3 ssh_dictionary_attack.py ../data/ssh-lab-wordlist.txt
```

Expected output shape:

```
Target: labstudent@lab01-ssh-target:22
Attempts: N
Elapsed: N.NNN seconds
Result: candidate found
```

The program intentionally does not print the matching candidate. Record only the attempt count, elapsed time, and success status in your report.

Step 5 - Observe and remove the target

In a second host terminal, inspect the local container logs:

```
docker compose -f docker/compose.yml --profile lab01-ssh \
  logs --tail 50 lab01-ssh-target
```

Compare the visible failed-login records with the offline attacks, which give no server-side signal. Exit the course shell, then remove the target:

```
exit
docker compose -f docker/compose.yml --profile lab01-ssh \
  rm --stop --force lab01-ssh-target
```

Offline versus online comparison

```
| Property | Hash-file attacks | SSH container attack |
| --- | --- | --- |
```

Guess reaches server	No	Yes
Server can log attempts	No	Yes
Network/handshake overhead	No	Yes
Rate limiting possible	Not at server	Yes
Target in this lab	Local CSV	Internal Compose service

Implementation guidance without the solution

- Use `csv.DictReader` so columns are addressed by their documented names.
- Normalize hexadecimal digests before comparison.
- Use `time.perf_counter()` around only the attack loop.
- Increment the counter exactly when SHA-256 is computed.
- Keep candidates as strings for reporting and encode them only when hashing.
- Return recovered values from the function; do not rely only on printed text.

Completion criteria and report evidence

You have completed Part 2 when both scripts run without `NotImplementedError`, return the required tuple, and `compare_cost.py` prints two rows without a division-by-zero error. Your report must include recovered fictional account names, both computation counts, elapsed time, guesses per second, and a written explanation of the scaling difference. Do not publish recovered candidates. For the SSH extension, also include the bounded attempt count, elapsed time, redacted server log evidence, and an online/offline comparison. Do not include the matching SSH password.

Checkpoint questions

1. Why can one unsalted candidate digest serve all accounts?
2. Why must salted records be checked separately?
3. Which computation count grows faster as the number of accounts increases?
4. Why does salt prevent shared precomputation without making a weak password strong?
5. Why can the SSH server rate-limit or log this attack, while the CSV targets cannot?
6. Which safety properties would be lost if the target hostname or Docker port were made configurable?